# parbench: Measuring How Coding-Agent Harnesses Handle $k$ Concurrent, Conflicting Tasks

Team *vam* — Mid-Submission Progress Report (Phase 2)

Vishesh Saraswat
Aviral Malhotra
Manas Mittal
IIIT Hyderabad
India

## Abstract

Parallel coding agents waste compute when tasks touch the same files: they re-explore the same context, overwrite each other and produce changes that no longer merge. No benchmark measures this beyond two tasks. We are building `parbench`, a pipeline that bundles $k$ tasks from CooperBench into one repository snapshot, runs them under different orchestration strategies and harnesses, grades every task on the final tree, and logs tokens, time and conflicts. This report covers the architecture, the decisions made, the experiments run so far (Gemini and GPT-OSS through Antigravity, Qwen2.5-7B through OpenCode, a toy pool), the problems we hit, and the remaining work. Results are preliminary: small $n$, one repeat.

## 1 Problem and Motivation

Coding agents are now used on several tasks at once, by fanning out to subagents or by running one agent per task. When the tasks overlap, uncoordinated agents duplicate exploration, overwrite one another and produce patches that conflict at merge time. CooperBench [1] showed that two cooperating agents do worse than one agent doing both tasks, but it covers only pairs, one harness and a hand-built dataset. No benchmark lets us ask how orchestration strategy, harness and model affect *correctness, cost and wasted effort* as the number of concurrent tasks $k$ grows. Our goal (Phase 1 proposal) is a benchmark and evaluation pipeline for exactly this, built on existing datasets and existing harnesses; we do not train a model or write a new agent framework.

**Related work.** CooperBench [1] evaluates two agents cooperating on one task with a hand-written dataset and a single harness. NP-Bench compares uncoordinated agents with up-front planning but contains only nine hand-built scenarios. Other studies report hidden costs of parallelism, such as workers reconciling inconsistent outputs, yet existing benchmarks either force sequential work, fix the number of peers, or cannot scale past two tasks with a controllable amount of conflict. Our additions are the $k$-task bundling, several harnesses and models on the same bundles, and cost and waste measured next to correctness.

## 2 System and Architecture

**Vocabulary.** A *task* is one CooperBench feature with hidden tests; a *pool* is the set of tasks sharing one repository at one base commit (CooperBench calls the pool a "task" and our task a "feature"); a *bundle* is $k$ tasks from one pool in a fixed seeded order; a *run* is bundle × strategy × harness × model × repeat. CooperBench has 30 pools, 199 tasks and 652 pairs; we use its curated `core` subset (10 pairs, 9 repositories, four languages).

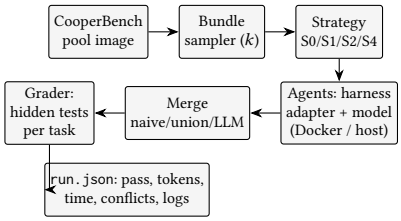



**Figure 1: The `parbench` pipeline. Every agent starts from a clean copy of the pool image with later upstream history removed.**

**Pipeline** (`src/parbench`, about 2.7k lines of Python, 34 commits from three contributors). (1) A bundle sampler builds $k$-task bundles. For $k > 2$ it extends each curated pair with more tasks from the same pool. (2) Each agent starts in a clean copy of the repository with the history after the base commit removed (CooperBench images contain the upstream future commits). (3) A strategy runs the agents. (4) If an agent ends by asking a question it is resumed with "No human is available; use your best judgement" (counted as a nudge). (5) We collect each agent's diff ourselves. (6) A grader applies the final tree, as one diff against the base, and runs each task's hidden tests with the pool image's own `runner.sh`; a task passes only if the runner exits 0 and at least one test passed. CooperBench's grader handles two features only, so ours handles any $k$. (7) A record per run stores correctness, tokens, wall-clock time, agent-busy time, conflicts, overlap and per-agent logs.

**Strategies.** S0 runs each task alone (the $k$=1 reference for retention). S1 gives one agent all $k$ tasks. S2 runs one agent per task in parallel and merges the patches in three modes: plain `git merge`, git's union driver, and an LLM resolver that is the same agent. S4 gives all tasks to a harness with its native subagents enabled, in a variant where the lead agent also does the merging in git worktrees. Prompt variants (plan first, other tasks visible) are in place; a planner-based strategy (S3) is not built yet.

**Harnesses.** Adapters exist for OpenCode, Pi, Claude Code, Codex CLI, Antigravity (`agy`) and Gemini CLI; each writes its own config, runs headless with JSON output, and a parser converts tokens to a common convention (input including cached tokens). Only OpenCode and `agy` have been checked against real output so far.

**Sandbox.** Agents run in Docker containers built from the CooperBench pool image, so they can run the project's own tests. A host

mode (temporary clone, no Docker) exists for agy and for OpenCode; a second no-Docker route was added using prepared host directories for use on a GPU machine.

## 3 Decisions Made

- **Build on CooperBench** (pinned submodule, commit `63b9d44`) and re-bundle it, instead of writing tasks by hand.
- **Strategies first**: S0, S1, S2, S4. The team's literature notes are treated as guidance, not a specification.
- **Docker sandbox per agent** and agents may run the repository's tests; hidden tests are never visible.
- **Fully autonomous agents**: no human in the loop, resumed with a fixed nudge, up to two times.
- **Compute**: local Ollama on a 24 GB Mac for smoke tests; a GPU machine with vLLM for the main OpenCode runs (run on a separate machine); Antigravity for hosted Gemini and GPT-OSS. The team accepted the terms-of-service risk of scripting agy on a personal account at low concurrency.
- **Fairness**: same model for agents, resolver and planner; same step limit; one seeded task order per bundle.
- **Everything is logged raw** (JSON from each harness), so parsers can be fixed and runs re-scored without re-running agents.

## 4 Experimental Setup

**Datasets.** (a) A toy pool (6 features of a small Python package, known conflict structure) to check the pipeline. (b) CooperBench `core`: five Python/Go pools so far (dspy 8563, go_chi 27, click 2800, jinja 1559 and 1621), $k$=2, one repeat. **Grader check.** For 41 real features in six pools, the tests fail on the base, the gold patch passes, and CooperBench's combined patch passes, on arm64 images; likewise for the six toy features. **Models and harnesses.** agy with Gemini 3.8 Flash (low) and GPT-OSS 120B; OpenCode with Qwen3.5-9B (Ollama, thinking off, 64K context) on the toy pool and with Qwen2.5-7B-Instruct (vLLM, one RTX 5000 Ada; partial) on the Python pools of `core`. **Metrics.** Task pass rate, bundle success, retention versus S0, pairwise conflict rate, overlap, tokens, wall-clock and agent-busy time, effective parallelism, nudges, and a validity flag. No dollar cost is computed for subscription models.

## 5 Results (Preliminary)

**Toy pool, OpenCode + Qwen3.5-9B (local).** All strategies pass all tasks: S0 $k$=2 (938 s), S1 (536 s), S2 (592 s, all three merge modes), S0 $k$=3 (1312 s). The bundle used had no conflicting pair, so the conflict path of the resolver was not exercised. These runs show the pipeline works end to end; they are not results.

**Antigravity, `core`.** Table 1 shows 21 valid runs. With Gemini, S1 (one agent, both tasks) passes slightly more than S0 (0.70 vs. 0.60, $n$=5) and, because the isolated reference runs two agents, uses 2.4× fewer input tokens (6.8M vs. 16.0M) and less than half the wall-clock time (554 s vs. 1350 s). S2 passes 0.83 ($n$=3) with a pairwise conflict rate of 0.33 and an effective parallelism of 1.56; S4 has one valid run so far. The differences are not statistically meaningful at this $n$. GPT-OSS results are dominated by service failures (below) and say little about the model. A single task costs 3–8M input tokens, far above our initial estimate.

**Table 1: agy on CooperBench `core` (5 pools, $k$=2, valid runs only, as of 6 Nov, 22:30 IST; runs still in progress). Task pass = mean fraction of the two tasks passing; Bundle = both pass; wall and tokens are medians per run.**

| Model | Strat. | $n$ | Task pass | Bundle | Wall (s) | In-tokens (M) |
|---|---|---|---|---|---|---|
| Gemini 3.8 Flash | S0 | 5 | 0.60 | 0.20 | 1350 | 16.0 |
| Gemini 3.8 Flash | S1 | 5 | 0.70 | 0.40 | 554 | 6.8 |
| Gemini 3.8 Flash | S2 | 3 | 0.83 | 0.67 | 643 | 8.7 |
| Gemini 3.8 Flash | S4 | 1 | 0.50 | 0.00 | 339 | 3.3 |
| GPT-OSS 120B | S0 | 3 | 0.33 | 0.00 | 1255 | 6.3 |
| GPT-OSS 120B | S1 | 2 | 0.00 | 0.00 | 357 | 2.3 |
| GPT-OSS 120B | S4 | 2 | 0.00 | 0.00 | 188 | 1.0 |

**Table 2: Task pass fraction per pool (valid runs; 1.0 = both tasks pass, – = no valid run yet).**

| | Gemini 3.8 Flash | | | | GPT-OSS 120B | | | |
|---|---|---|---|---|---|---|---|---|
| Pool | S0 | S1 | S2 | S4 | S0 | S1 | S2 | S4 |
| dspy 8563 | 0.5 | 0.5 | 0.5 | 0.5 | – | – | – | 0.0 |
| go_chi 27 | 0.5 | 0.5 | 1.0 | – | 0.5 | – | – | – |
| click 2800 | 0.5 | 0.5 | – | – | 0.0 | 0.0 | – | – |
| jinja 1559 | 0.5 | 1.0 | 1.0 | – | 0.5 | 0.0 | – | 0.0 |
| jinja 1621 | 1.0 | 1.0 | – | – | – | – | – | – |

**Per-pool view.** Table 2 shows the pattern behind the averages. With Gemini, the jinja 1621 pair passes both tasks in S0 and S1, while the dspy, go_chi and click pairs pass exactly one of two in S0 and S1: one task per pair is consistently out of reach for this setup, whatever the strategy. We have not yet inspected which task fails or why. Because the agents could not run the repository tests on the host, some failures may be unverified mistakes, not missing capability.

**OpenCode + Qwen2.5-7B (GPU machine, partial, Python pools).** Across S0, S1 and S2 ($n$=11, 11, 9) no task has passed (0.00, 95% CI up to 0.15–0.18), with and without a "small model" prompt. We cannot yet tell whether the model is too weak for these tasks (CooperBench reports weak results for small open models) or whether part of the no-Docker setup is at fault; this is the first thing to investigate.

## 6 Problems Faced

- **Dataset mismatch.** CooperBench's grader and runner handle two features and pass patches through one shell argument (large merged patches break); we wrote our own $k$-task grader and copy files with `docker cp`. Its images also contain future upstream commits, so we remove them before agents start.
- **Local serving.** Ollama's default 4K context silently truncates prompts; a thinking model took more than five minutes per call (fixed with reasoning off); Ollama serves requests one at a time on the Mac, so parallel timings measure a queue; a `keep_alive` bug broke the warm-up script.
- **Docker.** Docker Desktop had 3.8 GB of RAM, too little for large pools; its settings file is ignored, so the limit had to be raised in the UI.
- **Harness quirks.** OpenCode 2.x needs an explicit model map for custom providers (and differs from 1.x); agy rejects

prompts passed in the wrong flag order, denies tool calls headless unless allowed, has no switch to disable subagents, and its login cannot be carried into a container, so it runs on the host.

- **Safety and privacy.** A host-mode agent ran `env` and wrote the shell environment (session tokens) into a log. We redacted the logs before committing and now pass only an allow-list of environment variables.
- **Benchmark contamination.** `agy` agents used web search to look for the upstream pull request behind the task (*e.g.*, `site:github.com/pallets/click/pull/2800`). Deny rules did not block it, so we run `agy` with custom agents without web, browser and question tools, and an audit command flags any web use. 18 early runs were archived as contaminated and rerun.
- **Quota and capacity.** After about 216M Gemini input tokens the account hit a quota limit (429) and GPT-OSS returned 503 "no capacity" errors; 23 of 42 runs ended on a service error and are excluded and rerun. The runner now waits and resumes, and marks such runs invalid.
- **Scale.** The full benchmark at $k$=2 alone is 3,912 agent sessions (about 70,000 over all $k$), so we use curated subsets.
- **Process mistakes** we corrected: broad process-kill patterns that killed our own watchers, deleting live temporary clones, and editing prompt templates while an experiment was running.

## 7 Limitations

$n$ is tiny and each configuration has one repeat; confidence intervals are wide. `agy` runs on the host, so agents cannot run the repository's tests, which lowers absolute pass rates and makes them not directly comparable with containerised OpenCode runs; subagents cannot be disabled in `agy` S0–S2 apart from our custom agent. Model quotas and server capacity are outside our control. Only $k$=2 has been run, and only one harness has results on real pools with a model strong enough to pass tasks.

## 8 Milestones and Changes to the Proposal

**Achieved so far.** An end-to-end pipeline from bundle sampling to graded, logged runs; a $k$-task grader checked on 41 real CooperBench tasks and a toy pool; four strategies (S0, S1, S2 with three merge modes, S4) and prompt variants; six harness adapters, two verified on real output; a Docker sandbox and two no-Docker routes; first measurements with three harness/model setups; and tooling the proposal did not foresee: a web-use audit, a run-validity flag, automatic retry through quota errors, and raw-log redaction.

**Changes.** (1) *Models*: the course encourages local models, so the main OpenCode runs use open models served with vLLM, while hosted Gemini and GPT-OSS through Antigravity act as a stronger reference. (2) *Scale*: the full benchmark is not feasible (Section 6), so we evaluate the curated `core` subset first, with $k$=2, and move to $k$ up to 6 afterwards. (3) *Optional items*: bundling SWE-smith bugs to add an adjustable conflict-density dial remains "if time permits". (4) *Risk*: access to hosted models (quota, capacity, terms of use) turned out to be a schedule risk; we plan the final experiments around local models.

## 9 Remaining Work (to 30 November)

- Finish the `core` runs (S4 and the heavy pools: outlines, typst, tiktoken, llama_index, react-hook-form) and add repeats ($\geq$3) with confidence intervals.
- The main OpenCode experiments on a GPU machine with a stronger open model (Qwen 27B–35B class), and find out why Qwen2.5-7B passes nothing.
- $k = 3, 4, 6$ bundles and an analysis by conflict density using CooperBench's gold conflict report.
- Validate the Pi, Claude Code and Codex adapters on real output; cache S0 results across bundles.
- Our own contribution: a predict-then-schedule strategy (S3) that predicts each task's files, runs conflicting tasks one after another and the rest in parallel, evaluated with file-prediction recall and the cost saved.
- Report cost and resource use (tokens, GPU-hours) as in the proposal, and the final ACM-format report.

## 10 Use of AI Tools

Most of the pipeline code, documentation and this draft were written with Claude Code (Claude Opus 5.5 and Sonnet 5.5), including delegated research subagents; AI agents are also the subjects of the experiments (Gemini, GPT-OSS, Qwen via OpenCode and `agy`). Outputs were verified by running the grader on gold patches, dry-running experiment plans, inspecting run logs, and review of every commit. **[Team: attach prompt log and list how each person verified or changed AI-written parts.]**

## Repository

https://github.com/vishesh312/autose-project **[currently private: grant TA/instructor access or make public]**. The history has 34 commits from three contributors over 4–6 November (pipeline, adapters, documentation, and GPU-machine runs with automated result publishing). Start with `AGENTS.md`, `docs/findings.md` and `docs/core_subset_plan.md`.

## References

[1] CooperBench: benchmarking coding-agent cooperation. arXiv:2601.13295, 2026 (code: github.com/cooperbench/CooperBench).